

Aprendizaje Profundo

Facultad de Ingeniería
Universidad Popular del
Cesar



Dr. José Ramón Iglesias

DSP-ASIC BUILDER GROUP

Director Semillero TRIAC

Ingeniería Electrónica

Universidad Popular del Cesar

TRANSFER LEARNING

GENERATIVE ADVERSARIAL NETWORKS

. Transfer learning

- Ventajas
- Estrategias
- Ejemplo en colab

. Generative adversarial networks

- Introducción
- Usos
- Ejemplo en colab

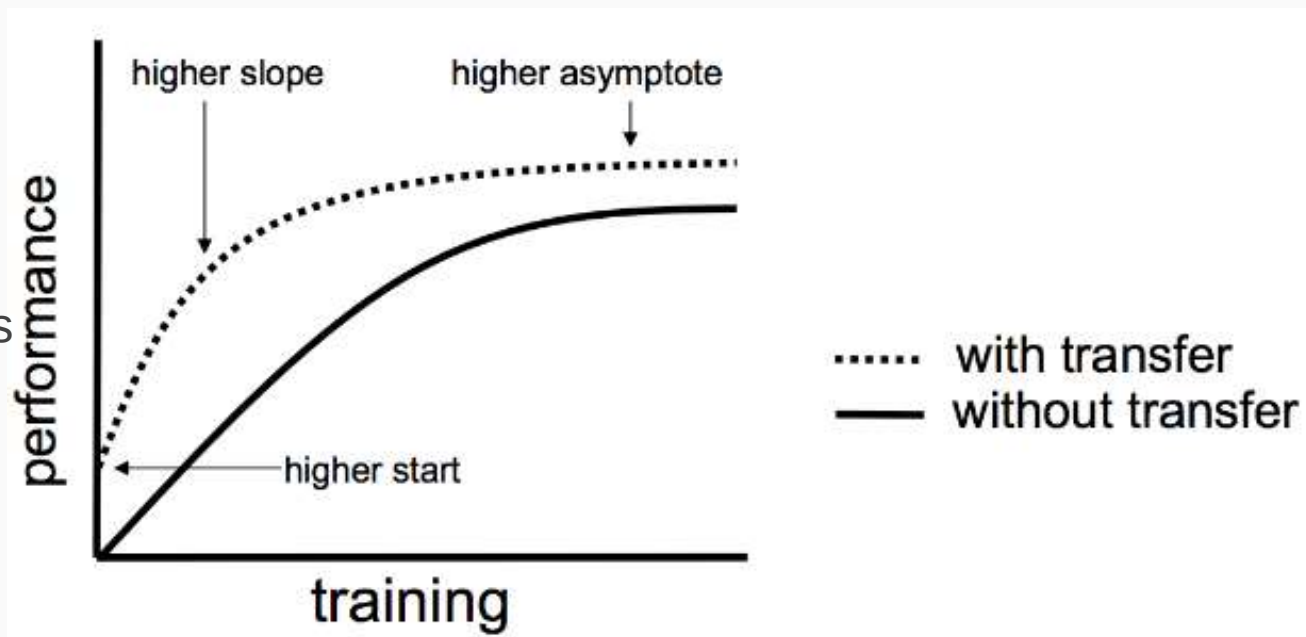
Transfer Learning

- No se suele diseñar y entrenar un modelo desde **CERO**
- Se emplean **modelos existentes y probados** con sus parámetros ya entrenados.
- Normalmente, los modelos que se toma de “base” cumplen una **tarea genérica**.
- Al modelo “base” se le hacen los ajuste necesarios para la nueva **tarea específica** que deben cumplir.

Transfer Learning

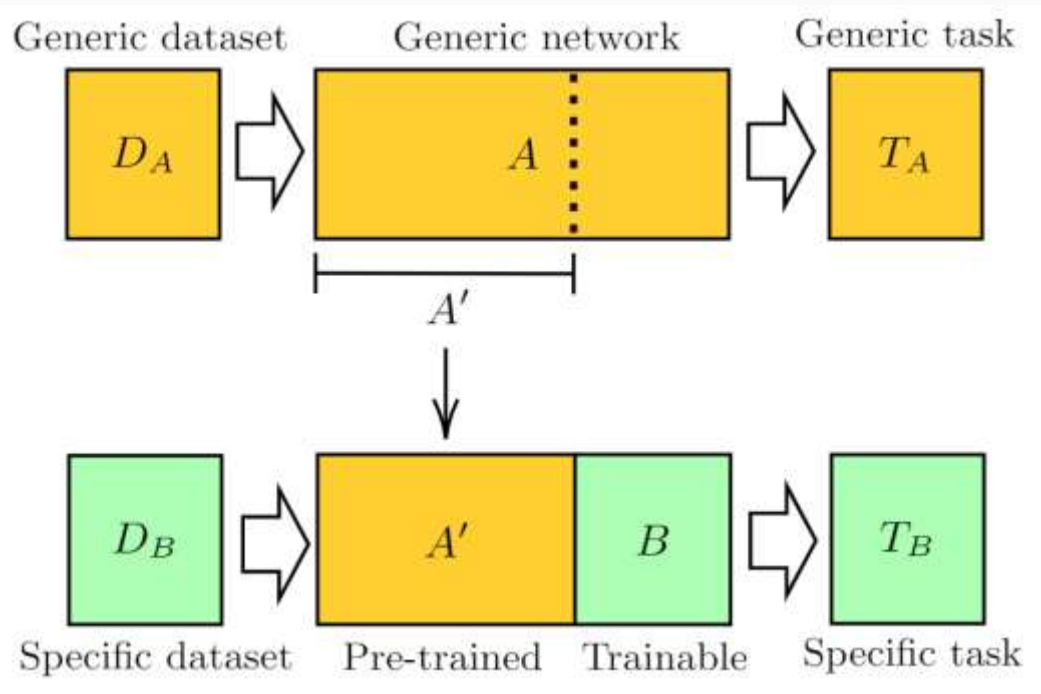
Ventajas:

- pocos datos
- pre-trained models
- pre-trained embeddings
- Simulations
- Cambio de dominio



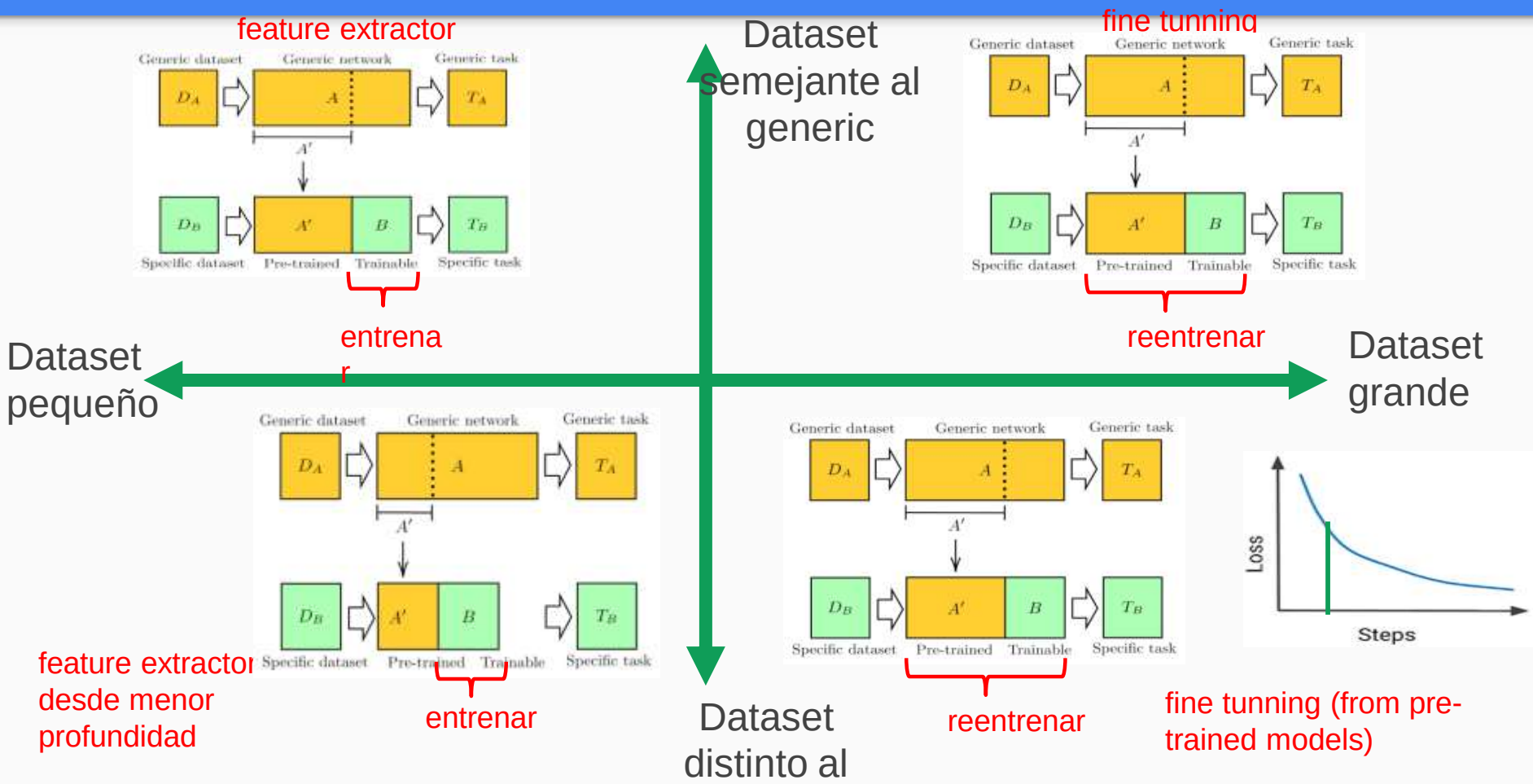
Transfer Learning

Adaptación de
modelo base
para cumplir la
tarea específica



Se reentrena la nueva arquitectura con el dataset específico bajo la tarea específica a

Transfer Learning - ¿Qué estrategia usar?



Transfer Learning - ejemplo código

Adaptación de modelo base para cumplir la tarea específica

Generic
Dataset:

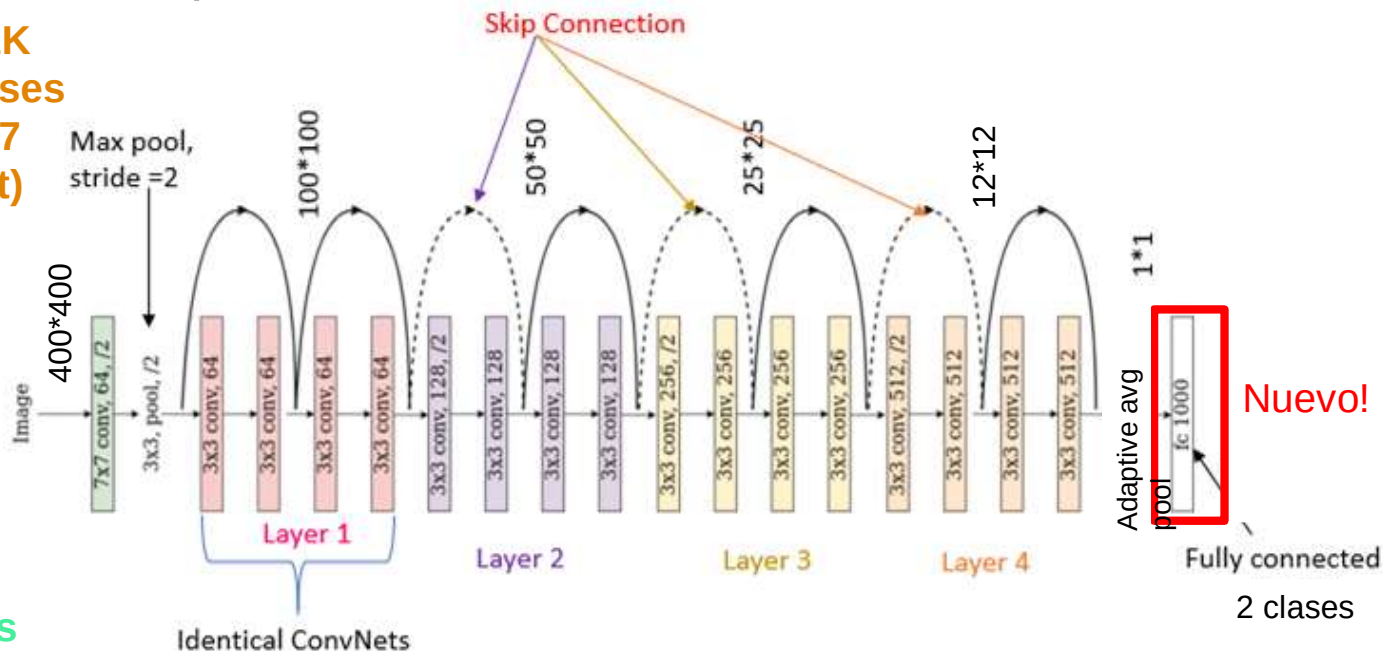
ImageNet 1K

- 1000 clases
- 1.281.167 (train set)
- RGB

Specific
Dataset:

Bees & ants

- 2 clases
- 120 (train set)
- RGB

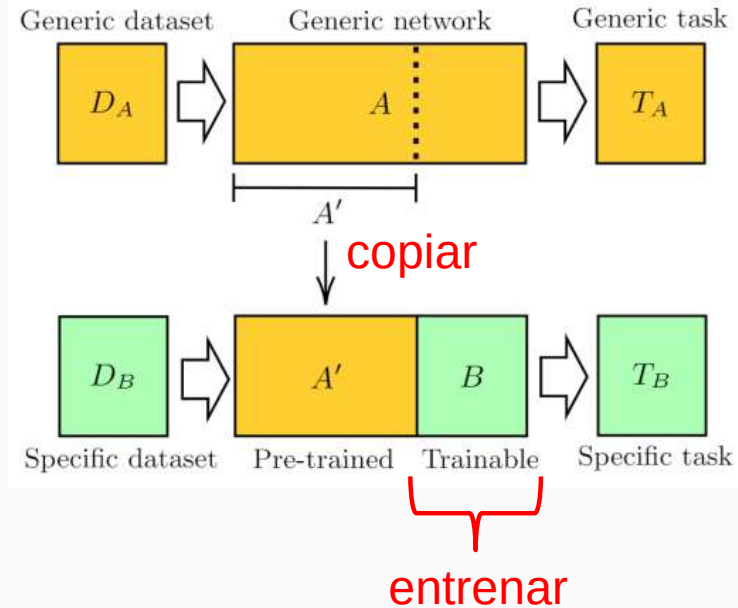


ResNet-18 Architecture

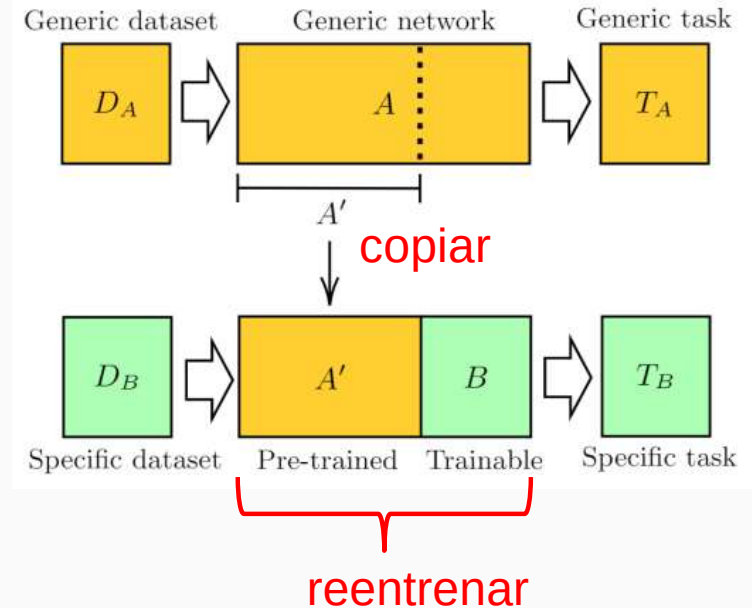
Transfer Learning - ejemplo codigo

VerCodigo

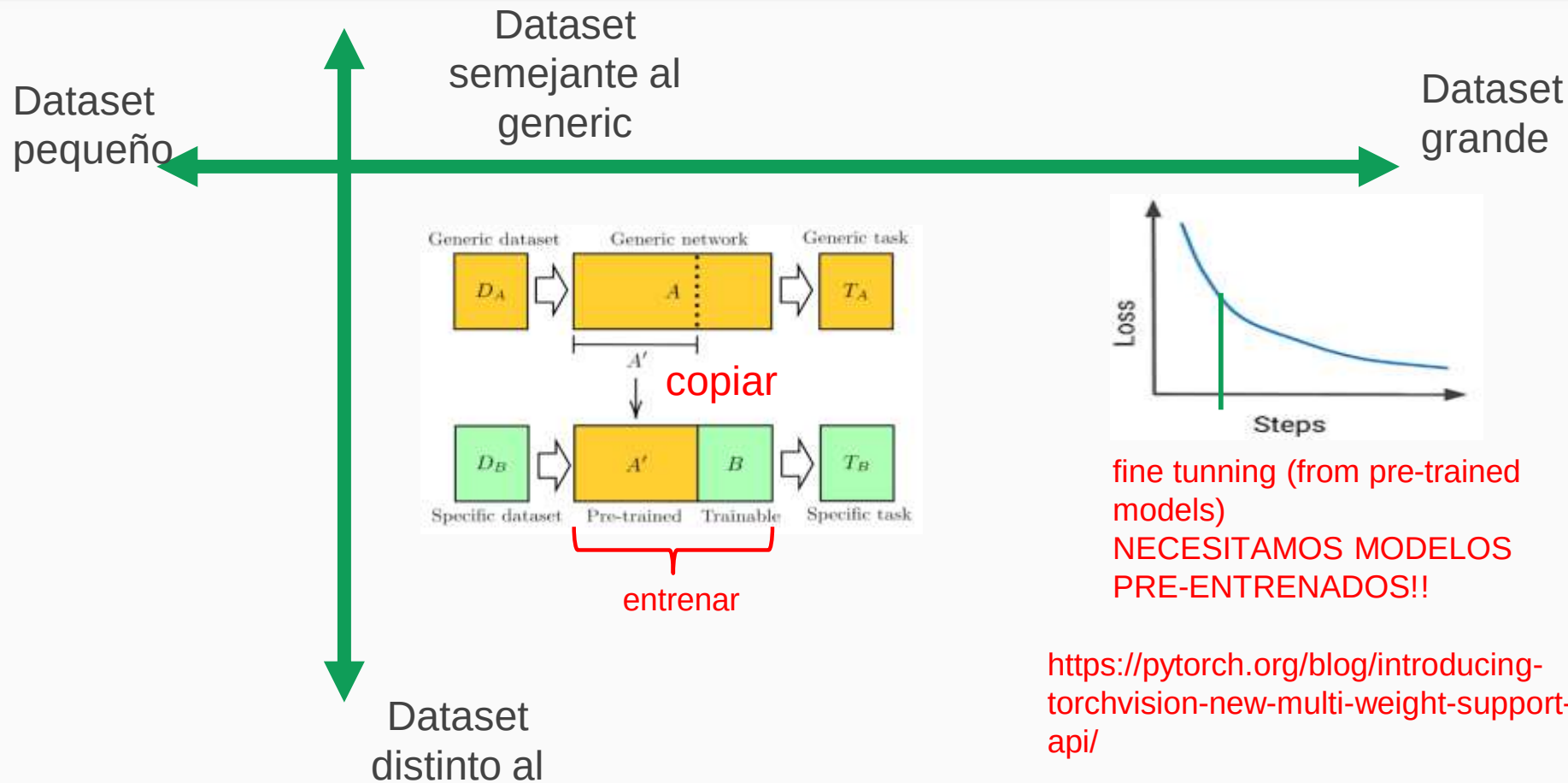
Feature extractor



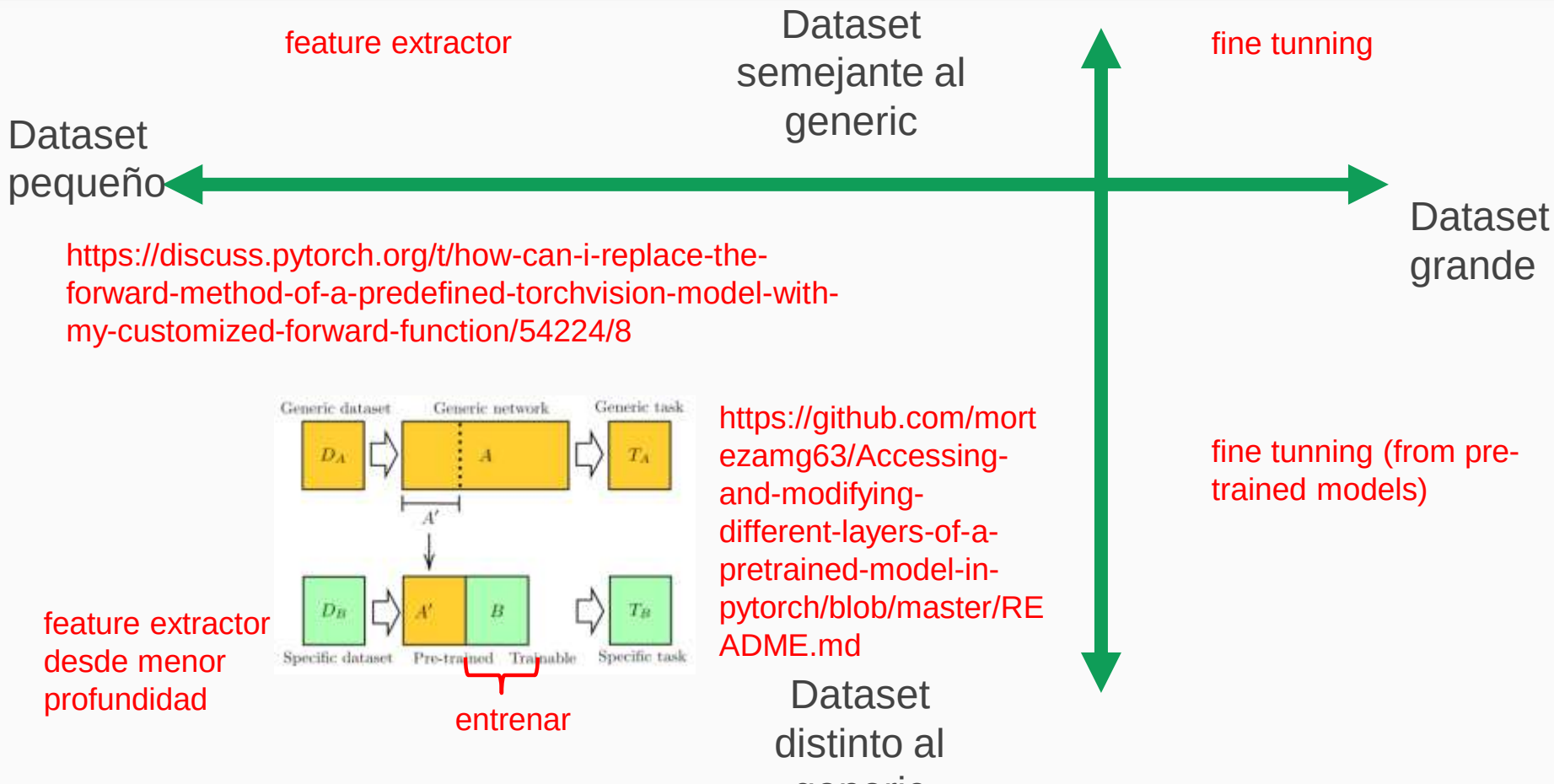
Fine tuning



Transfer Learning - ¿Qué estrategia usar?



Transfer Learning - ¿Qué estrategia usar?



¡Un merecido descanso!



Generative Adversarial Network (GAN)

Configuración de DL que busca **aprender la distribución de prob de los datos de entrenamiento** para poder **generar nuevos datos a partir de esa distribución**.

Se logra entrenando 2 modelos compitiendo:

G → generador que busca generar datos “falsos”

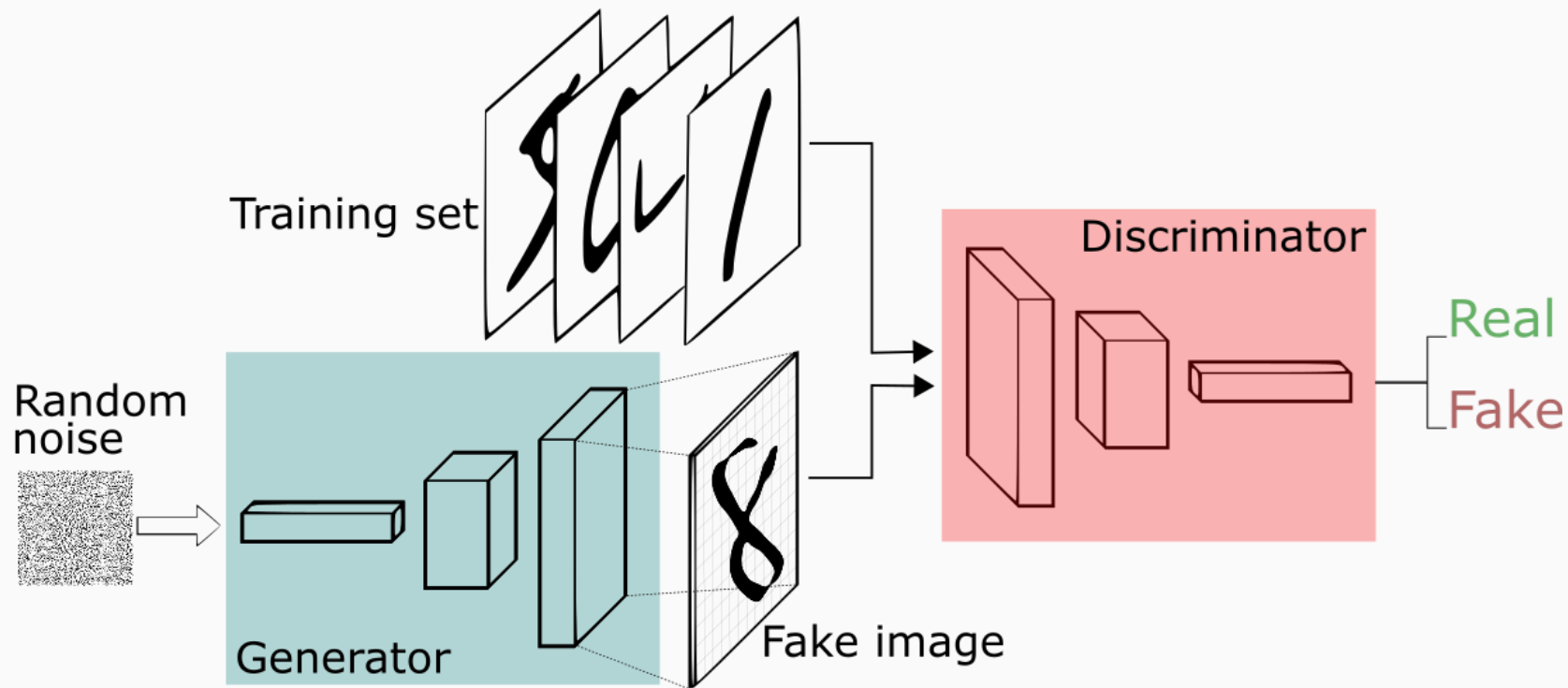
D → discriminador que busca identificar datos “verdaderos” o “falsos”

Cuando el sistema “converge” el **D** no es capaz de distinguir datos reales de falsos (50% de real y 50% de falso)

Ese punto de convergencia es teórico y aún no se ha podido

Generative Adversarial Network (GAN)

2 redes neuronales enfrentadas: **Generador** - **Discriminador**



Generative Adversarial Network (GAN)

2 redes neuronales enfrentadas:

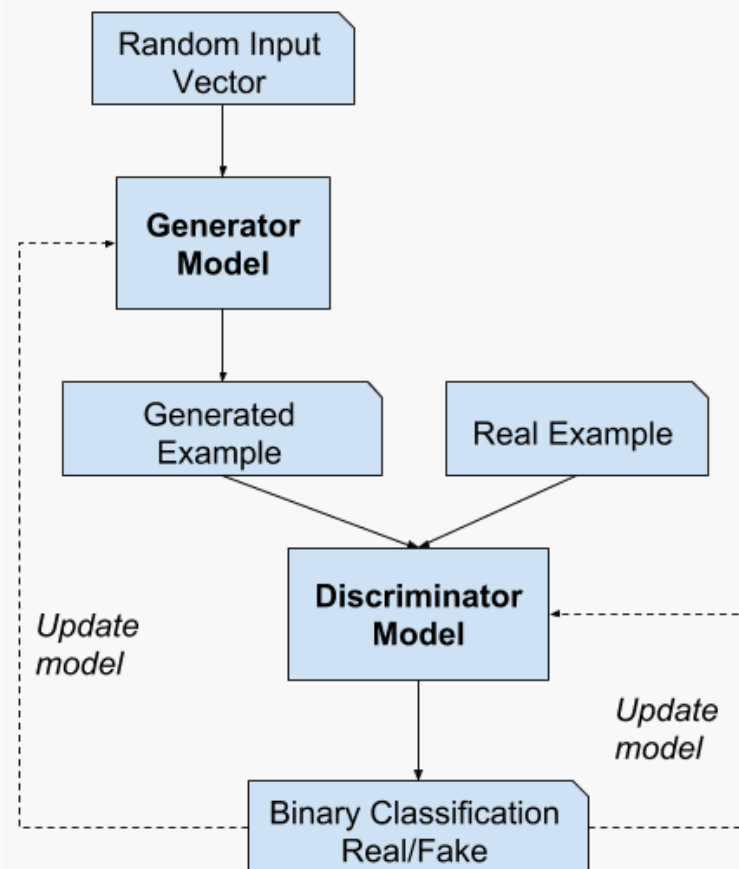
Generador - **Discriminador**

G → se entrena para que **D** falle

D → se entrena para no fallar

G → se entrena de forma indirecta
("supervisada"... pero de distinta forma)

D → determina si una muestra es real **1** o falsa **0**



Generative Adversarial Network (GAN) – Función de costo

G – generator (NN)

D – discriminator (NN)

z – vector aleatorio

x – vector muestra (real)

$D(x)=1$ si x es real

$D(x)=0$ si x es falsa

$G(z)$ – muestra falsa

El objetivo de D es
distinguir Real vs Fake

$$L_D = \sum_{batch} BCE(D(x), 1) + BCE(D(G(z)), 0)$$

$$L_G = \sum_{batch} BCE(D(G(z)), 1)$$



El objetivo de
G es “engañar”
a D

Generative Adversarial Network (GAN) – Función de costo

En teoría, en el equilibrio ideal de una GAN es:

- $D(x) = 0.5$
- $D(G(z)) = 0.5$

porque el discriminador ya no puede distinguir reales de falsas mejor que al azar.

Entonces, si $D(x) = 0.5$ y $D(G(z)) = 0.5$:

- $BCE(0.5, 1) = -\log(0.5) = 0.693$
- $BCE(0.5, 0) = -\log(0.5) = 0.693$

por lo tanto, las pérdidas de equilibrio rondan:

- $L_D = 0.693 + 0.693 = 1.386$
- $L_G = 0.693$

El objetivo de D es distinguir Real vs Fake

$$L_D = \sum_{batch} BCE(D(x), 1) + BCE(D(G(z)), 0)$$

$$L_G = \sum_{batch} BCE(D(G(z)), 1)$$



El objetivo de G es “engañar” a D

Generative Adversarial Network (GAN) - Entrenamiento

Entrena GANs es actualmente una especie de “trabajo de artesano”

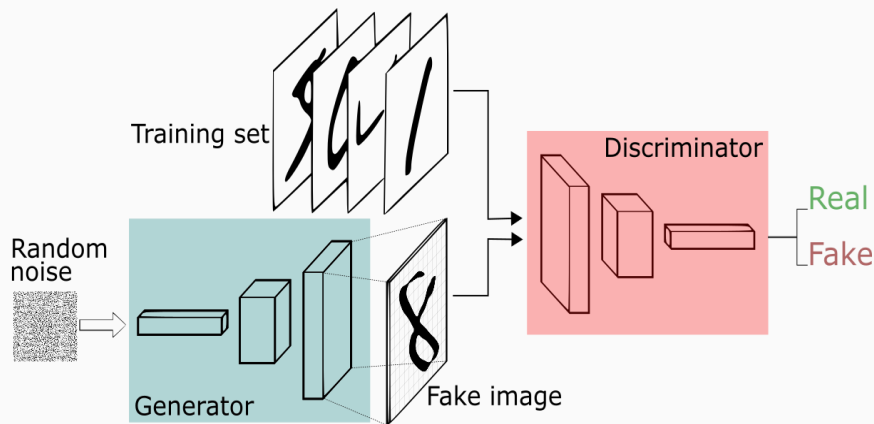
Algunos trucos son:

- TanH como output del generador.
- Poner etiquetas reales a muestras falsas para entrenar generador.
- Usar batch normalization y minibatch.
- Evitar Maxpool, usar average pool o hacer downsampling con el stride de conv.
- Evitar usar ReLu, usar LeakyReLu.
- Usar Adam como optimizador, $\beta_1 = 0,5$, $\text{lr}=0,0002$.
- Usar label smoothing (en lugar de 1 – 0; usar 0,9 – 0,1 para las clases real-fake).

Generative Adversarial Network (GAN) - Implementación

Implementación de GANs para generar número nuevos de MNIST

- VanillaGAN → GAN implementada con MLP
- DCGAN → GAN implementada con capas de convolución



Generative Adversarial Network (GAN) - Experimentos

Experimentos propuestos (tarea para el hogar)

- exp1 – Variar tamaño de `n_noise` (tamaño ruido input generador)
- exp2 – Variar `lr` y `beta1` (momentum) del optimizer adam
- exp3 – Varia `batch_size` y `n_critic` (entrenar mas veces D que G)
- exp4 – Entrenamiento extendido (200 epochs)
- exp5 – scheduler `lr` a D y G, para evitar sobreaprendizaje del G

Generative Adversarial Network (GAN) - Experimentos

exp1 – Variar tamaño de n_noise (tamaño ruido input generador)



$n_noise =$
10



$n_noise =$
50



$n_noise =$
100



$n_noise =$
200

Generative Adversarial Network (GAN) - Experimentos

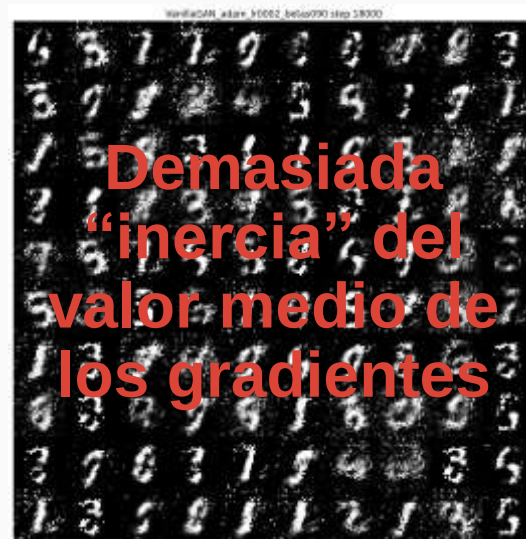
exp2 – Variar lr y beta1 (momentum) del optimizer adam (media de los gradientes)



lr = 0,001
beta = 0,50



lr = 0,0002
beta = 0,50



lr = 0,0002
beta = 0,90

Generative Adversarial Network (GAN) - Experimentos

exp3 – Varia batch_size y n_critic (entrenar mas veces D que

c)



batch= 32
n_critic = 1



batch= 64
n_critic = 1



batch= 128
n_critic = 1

Generative Adversarial Network (GAN) - Experimentos

exp3 – Varia batch_size y n_critic (entrenar mas veces D que

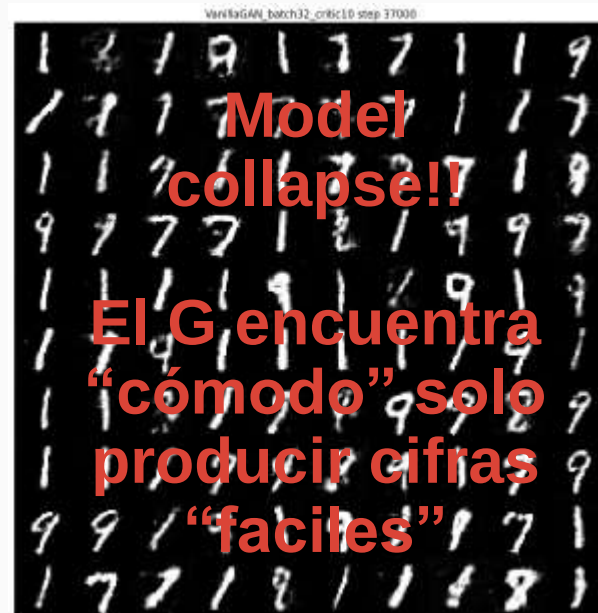
G)



batch= 32
n_critic= 1



batch= 32
n_critic = 5



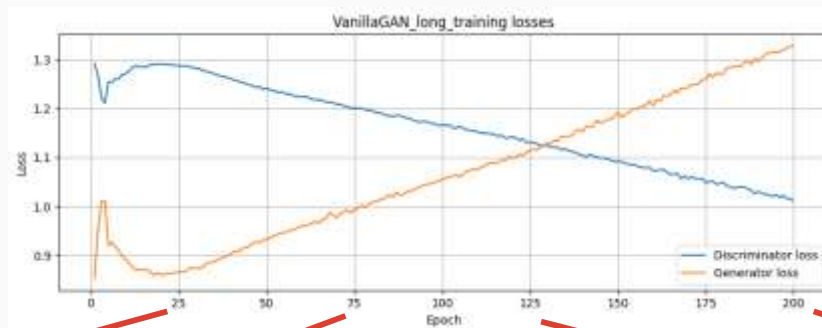
**Model
collapse!!**
**El G encuentra
“cómodo” solo
producir cifras
“faciles”**

batch= 32
n_critic = 10

Generative Adversarial Network (GAN) - Experimentos

exp4 – Entrenamiento extendido (200 epochs)

D sigue mejorando como clasificador real/falso, pero G ya alcanzó un nivel visual aceptable y no mejora en la métrica adversaria al mismo ritmo.

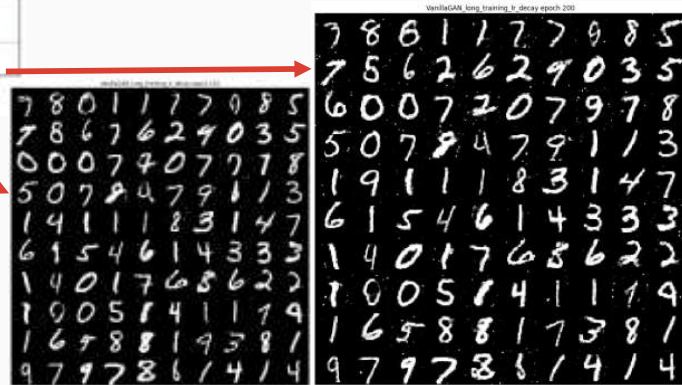
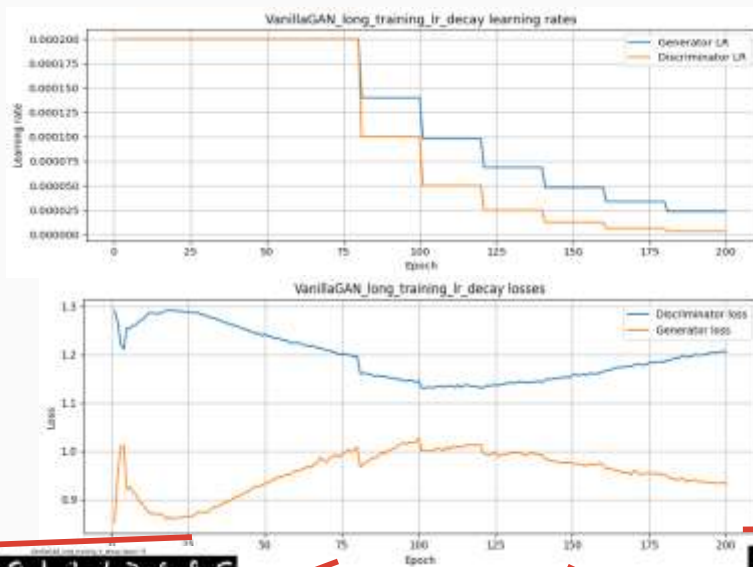


- al principio ambos mejoran rápido
- después D aprende una frontera de decisión bastante buena
- más adelante G ya produce dígitos razonables, pero corregir defectos finos es mucho más difícil
- entonces D sigue mejorando un poco, y ese pequeño margen ya alcanza para hacer subir G_loss .



Generative Adversarial Network (GAN) - Experimentos

exp5 – scheduler lr a D y G, para evitar sobreaprendizaje del G



Generative Adversarial Network (GAN) - Experimentos



exp4 vs exp5

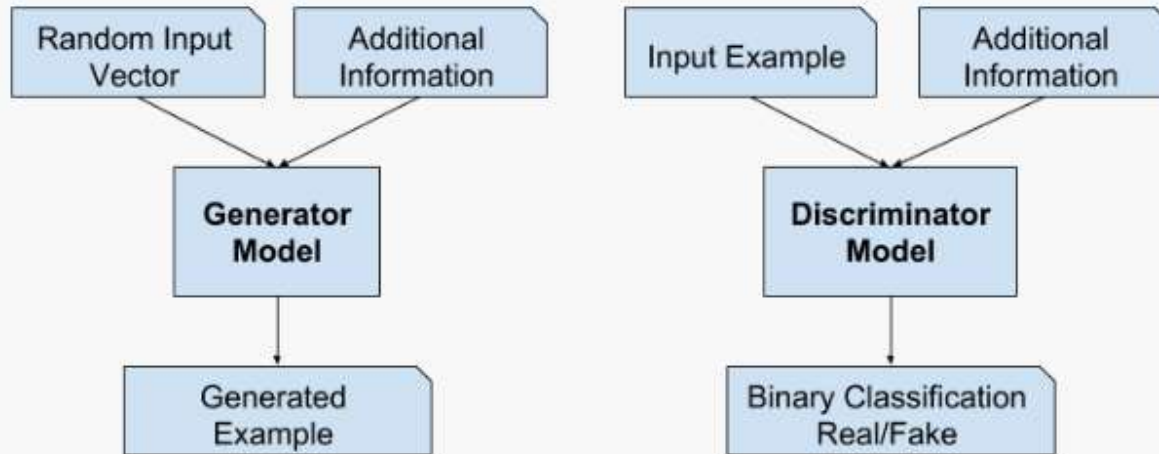


Que el discriminador esté alrededor de 0.5 quiere decir: “ya no distingue con mucha confianza entre real y falso”. Eso es estabilidad adversaria, no garantía de mejores dígitos.

Generative Adversarial Network (GAN) - Implementación

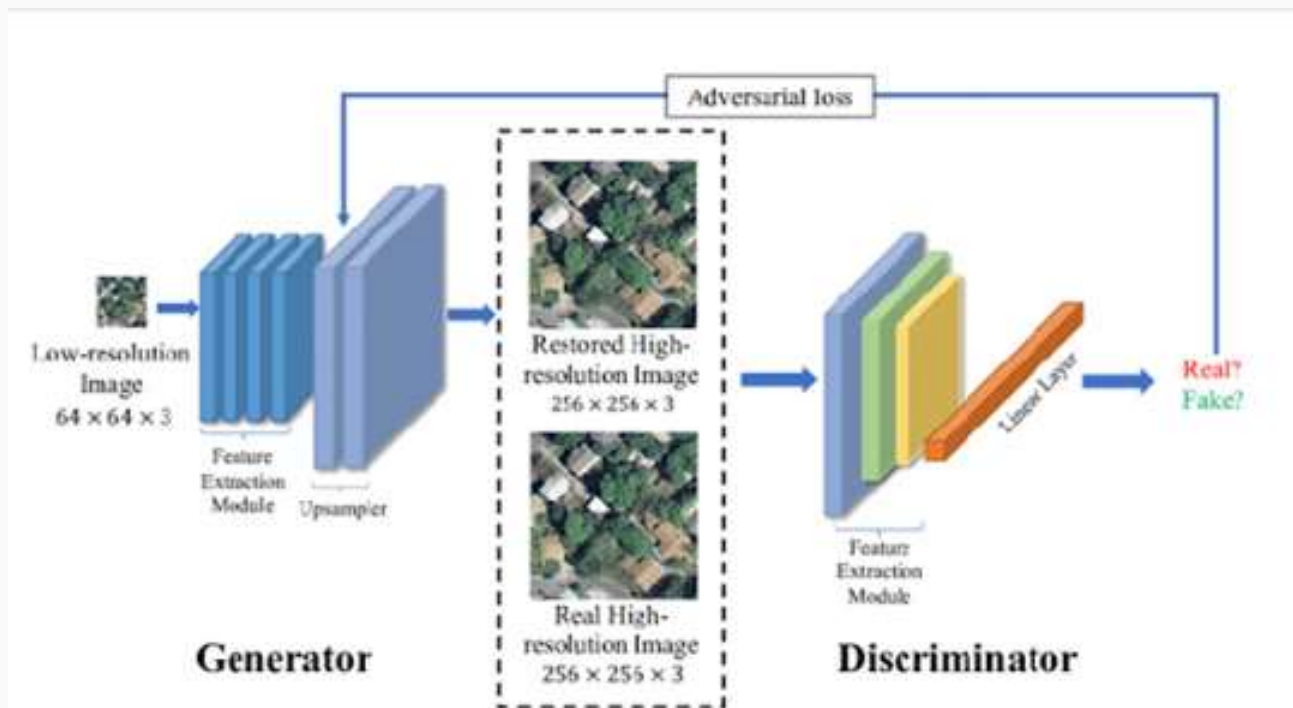
Conditionals GANs

Se le pasa un 'label' para que genere algo bajo ese 'label'



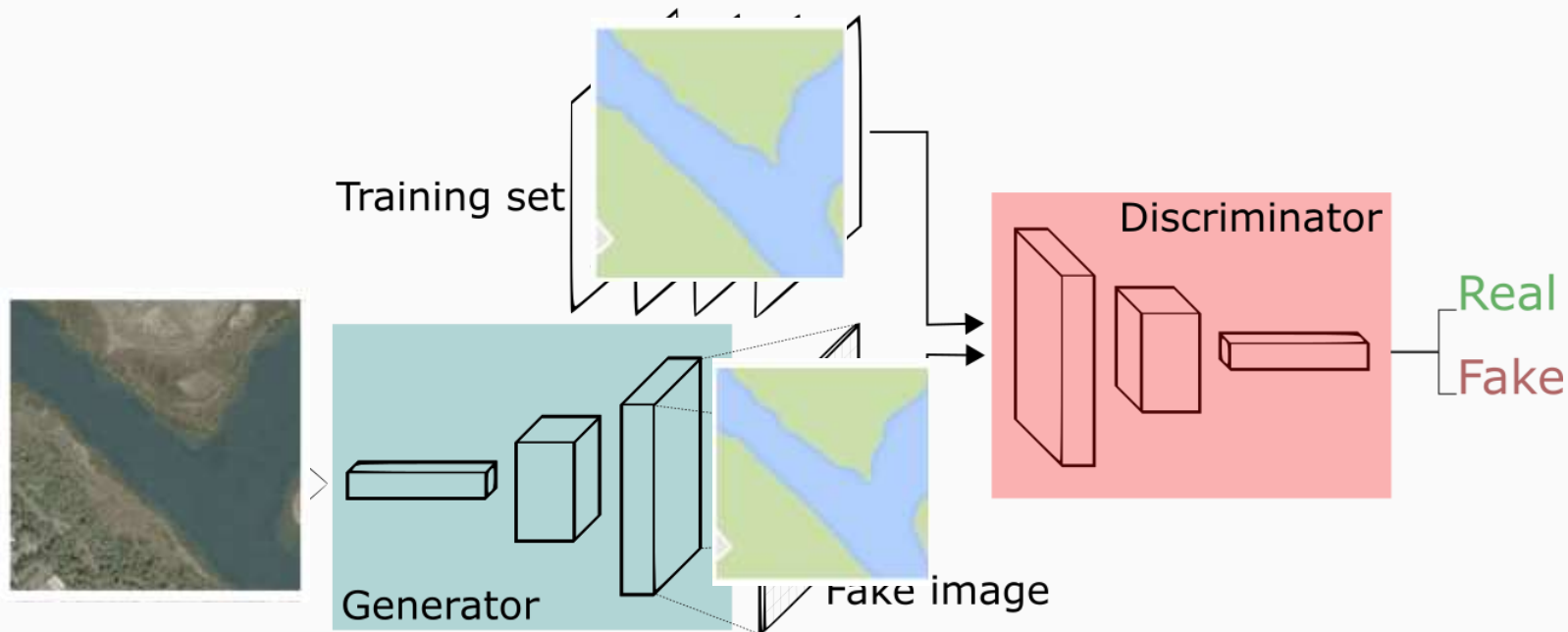
Generative Adversarial Network (GAN) – Otros ejemplos

Super resolution GANs



Generative Adversarial Network (GAN) – Otros ejemplos

Remote Sensing Image to Map Translation GANs



Generative Adversarial Network (GAN)

GANs

- ver código

<https://github.com/Yangyangii/GAN-Tutorial>

How to train

GANs:

[https://neptune.ai/blog/gan-loss-](https://neptune.ai/blog/gan-loss-functions)

[functions
https://arxiv.org/abs/1606.03498](https://arxiv.org/abs/1606.03498)

<https://github.com/soumith/ganhacks>

<https://dropsofai.com/best-practices-for-training-stable-gans/>

Mas sobre GANs:

https://docs.pytorch.org/tutorials/beginner/dc_gan_faces_tutorial.html

<https://github.com/hindupuravinash/the-gan-zoo>

<https://github.com/nashory/gans-awesome-applications>